Министерство образования Республики Беларусь

Учреждение образования
БЕЛОРУССКИЙ ГОСУДАРСТВЕННЫЙ УНИВЕРСИТЕТ
ИНФОРМАТИКИ И РАДИОЭЛЕКТРОНИКИ

Факультет информационных технологий и управления
Кафедра интеллектуальных информационных технологий

ОТЧЁТ

по лабораторной работе № 1
по дисциплине

ЛОГИЧЕСКИЕ ОСНОВЫ ИНТЕЛЛЕКТУАЛЬНЫХ СИСТЕМ

Студент гр. 421702:
Сайковский А. В.

Проверил:
В. П. Ивашенко

Минск 2026

ХОД ВЫПОЛНЕНИЯ ЛАБОРАТОРНОЙ РАБОТЫ

Вариант 1

Тема: Программирование операций обработки и преобразования формул прикладных и неклассических логик.
Цель: Приобрести навыки программирования алгоритмов обработки данных в неклассических логических моделях решения задач.
Задание: Реализовать прямой нечёткий логический вывод с использованием треугольной нормы min и нечёткой импликации Гёделя.

Описание работы

Разработана программа на языке Python, выполняющая прямой нечёткий логический вывод по фактам и правилам из текстового файла. Факты задаются конечными нечёткими множествами, а правила имеют форму `A(x) ~> B(y)` (в файле фактически используется слитная запись `~>`). Для каждого применимого правила строится нечёткое бинарное отношение по импликации Гёделя, после чего вычисляется max-min композиция с входным фактом. Новые факты добавляются в базу до прекращения её расширения.

При реализации использованы стандартные средства Python, модули `re` и `pathlib`, списки, кортежи, множества и словари. Совместимость фактов определяется совпадением наборов элементов их универсумов. Уже известные результаты обнаруживаются сравнением списков пар, поэтому при таком сравнении учитывается порядок элементов. Предполагается, что входные степени принадлежности находятся в диапазоне $[0, 1]$, а элементы внутри множества не повторяются; отдельная проверка этих условий в текущей реализации отсутствует.

Теоретические сведения

Нечёткое множество A на универсуме X определяется функцией принадлежности $\mu_A : X \to [0, 1]$. Значение $\mu_A(x)$ задаёт степень принадлежности элемента x множеству A. Для конечного универсума множество записывается в виде

$$A = \{\langle x_1, \mu_A(x_1)\rangle, \ldots, \langle x_n, \mu_A(x_n)\rangle\}. \tag{1}$$

Нечёткий предикат — предикат, истинностное значение которого при-

надлежит интервалу $[0, 1]$. Нормальным называется нечёткое множество высоты $h(A) = 1$, где $h(A) = \sup_x \mu_A(x)$. Если $h(A) < 1$, множество является субнормальным.

Треугольная норма (t-норма) — бинарная операция на $[0, 1]$, обобщающая конъюнкцию. В данной работе используется

$$T(a, b) = \min(a, b). \tag{2}$$

Импликация Гёделя задаётся выражением

$$I_G(a, b) = \begin{cases} 1, & a \le b, \\ b, & a > b. \end{cases} \tag{3}$$

Нечёткое бинарное отношение, соответствующее правилу $A(x) \to B(y)$, определяется на $X \times Y$ следующим образом:

$$R(x, y) = I_G\big(\mu_A(x), \mu_B(y)\big). \tag{4}$$

Обобщённый modus ponens позволяет по входному факту $A'(x)$ и правилу $A(x) \to B(y)$ получить новое заключение $B'(y)$. Используется sup-min композиция, которая на конечном универсуме принимает вид

$$\boxed{\mu_{B'}(y) = \max_{x \in X} \min\big(\mu_{A'}(x), R(x, y)\big).} \tag{5}$$

Прямой нечёткий вывод — последовательное получение новых фактов из уже имеющихся фактов и нечётких правил. Результат каждого шага может использоваться в последующих шагах. Обход правил повторяется, пока количество различных фактов в базе не перестанет увеличиваться.

Формат базы знаний

Входные данные располагаются в текстовом файле `input_original.txt`, который считывается функцией `main()` программы `lab1_refactored.py`. Каждое определение нечёткого множества и каждое правило записывается на отдельной строке. Определения задаются именем и списком пар `<элемент, степень>`; ниже записываются правила со знаком `~>`. Допускаются пустые строки. Следующая упрощённая грамматика описывает основные конструкции формата:

```
<база знаний> ::= <список фактов> <список правил>
<список фактов> ::= <факт> | <факт> <список фактов>
<список правил> ::= <правило> | <правило> <список правил>
<факт> ::= <имя> = { <список пар> }
<список пар> ::= <пара> | <пара>, <список пар>
<пара> ::= < <элемент>, <степень> >
<правило> ::= <имя>(<переменная>) ~> <имя>(<переменная>)
<степень> ::= действительное число от 0 до 1
<имя нечёткого предиката> ::= <имя>
<имя> ::= <буква> | <буква><символ>
<символ> ::= <буква> | <цифра>
<буква> ::= A | ... | Z | a | ... | z
<цифра> ::= 0 | ... | 9
```

Имя состоит из одной латинской буквы либо из буквы и ещё одной буквы или цифры. Регистр учитывается: например, допустимы `A`, `A1`, `AB`, `A0`, `b0`. В программе используется шаблон `[A-Za-z][A-Za-z0-9]?`. Регулярные выражения фактов и правил привязаны к границам строк, чтобы имена большей длины не распознавались частично. Обозначения `_1`, `_2` и далее автоматически создаются при выводе и не являются именами входных предикатов.

Содержимое файла `input_original.txt`:

```
A = {<x1, 0.0>, <x2, 0.3>, <x3, 1.0>, <x4, 0.5>}
B = {<y1, 0.4>, <y2, 0.0>, <y3, 0.3>, <y4, 1.0>}
C = {<x1, 0.7>, <x2, 1.0>, <x3, 0.0>, <x4, 0.2>}

A(x) ~> B(y)
C(x) ~> A(y)
B(x) ~> C(y)
```

Структуры данных и функции программы

Словарь `facts` сопоставляет имени нечёткого множества список пар `(element, degree)`. Список `rules` хранит пары имён посылки и заключения. Бинарное нечёткое отношение представляется списком троек `(x, y, implication_degree)`.

- `read_knowledge_base(filename)` — считывает факты и правила с помощью регулярных выражений и преобразует степени принадлежности в числа;
- `build_godel_relation(antecedent, consequent)` — строит отношение по формуле (**??**);
- `max_min_composition(observation, relation)` — вычисляет заключение по формуле (**??**);
- `apply_fuzzy_rule(...)` — выполняет один шаг нечёткого вывода;
- `find_compatible_facts(...)` — находит факты, имеющие тот же набор элементов, что и посылка;
- `format_fuzzy_set(...)` — создаёт текстовую запись множества для вывода;
- `forward_fuzzy_inference(...)` — организует полный прямой вывод до отсутствия новых фактов;
- `main()` — запускает считывание и выводит результаты.

Схемы алгоритмов программы

Блок-схемы из архива исходных схем иллюстрируют основные этапы алгоритма. В текущей реализации проверка наличия полученного множества выполняется в `forward_fuzzy_inference()` сравнением списков пар. Считывание базы и форматирование результатов описаны текстом.

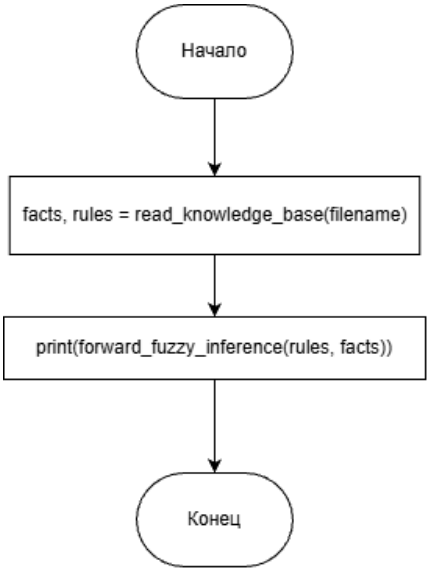


Рис. 1 Блок-схема функции `main()`

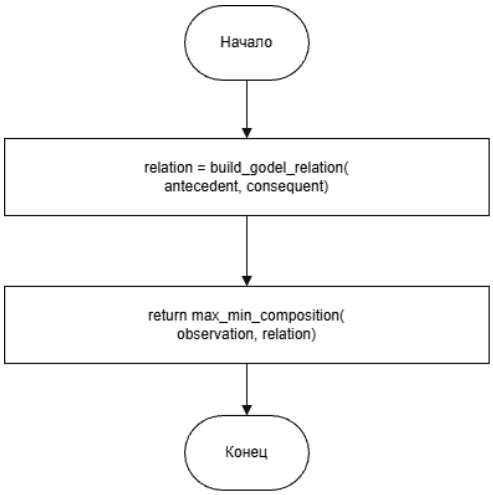


Рис. 2 Блок-схема функции `apply_fuzzy_rule()`

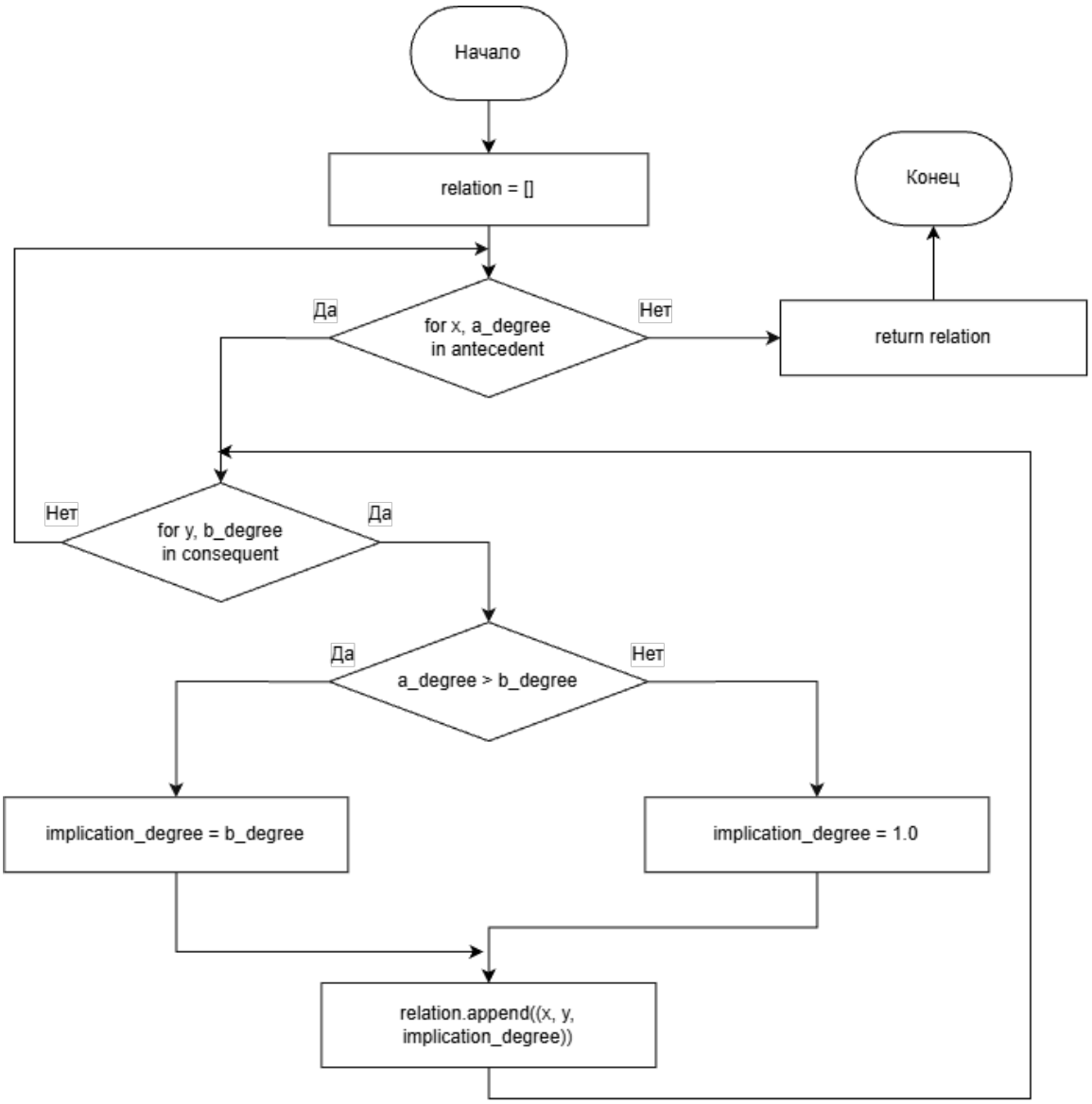


Рис. 3 Блок-схема функции `build_godel_relation()`

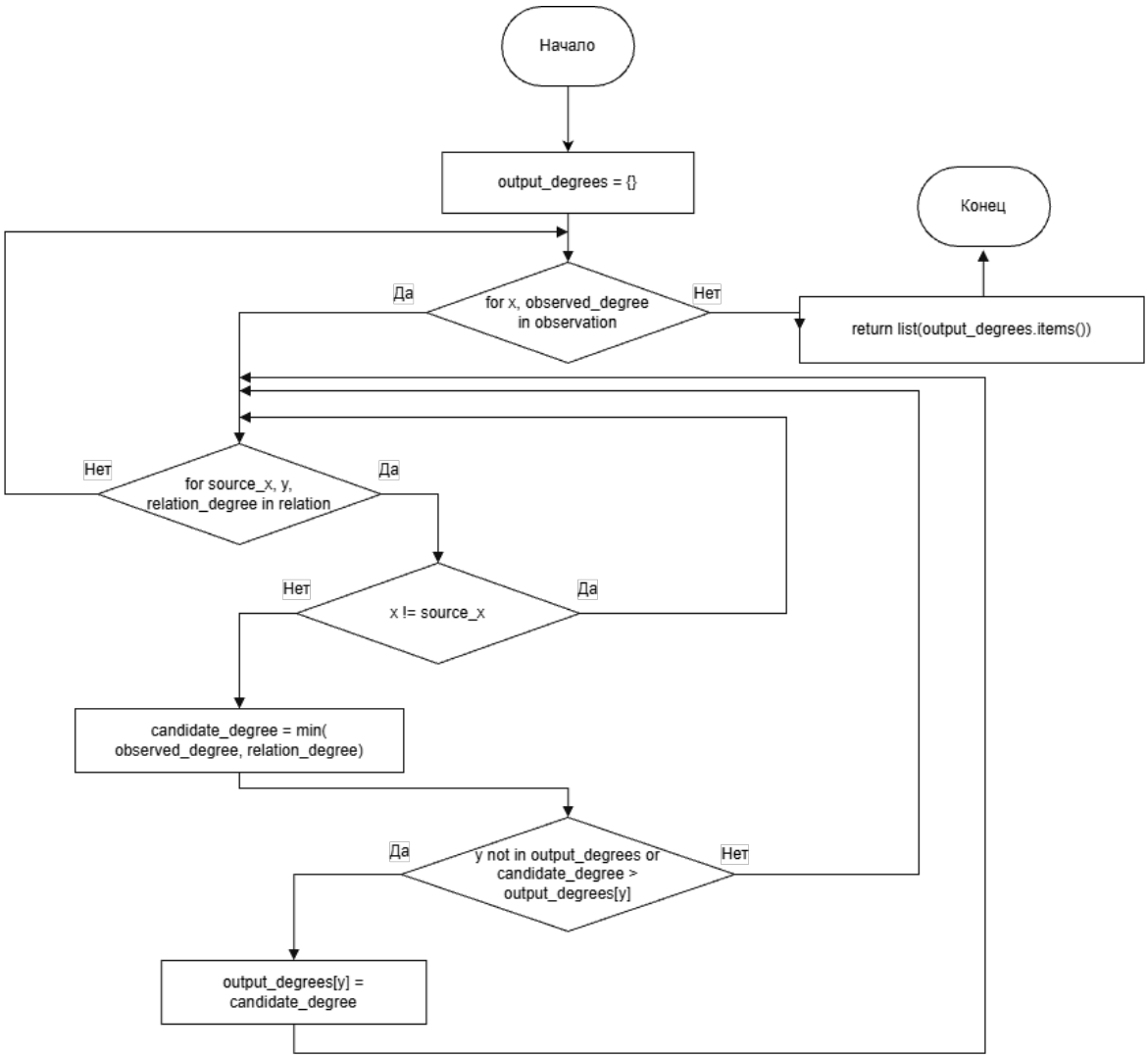


Рис. 4 Блок-схема функции `max_min_composition()`

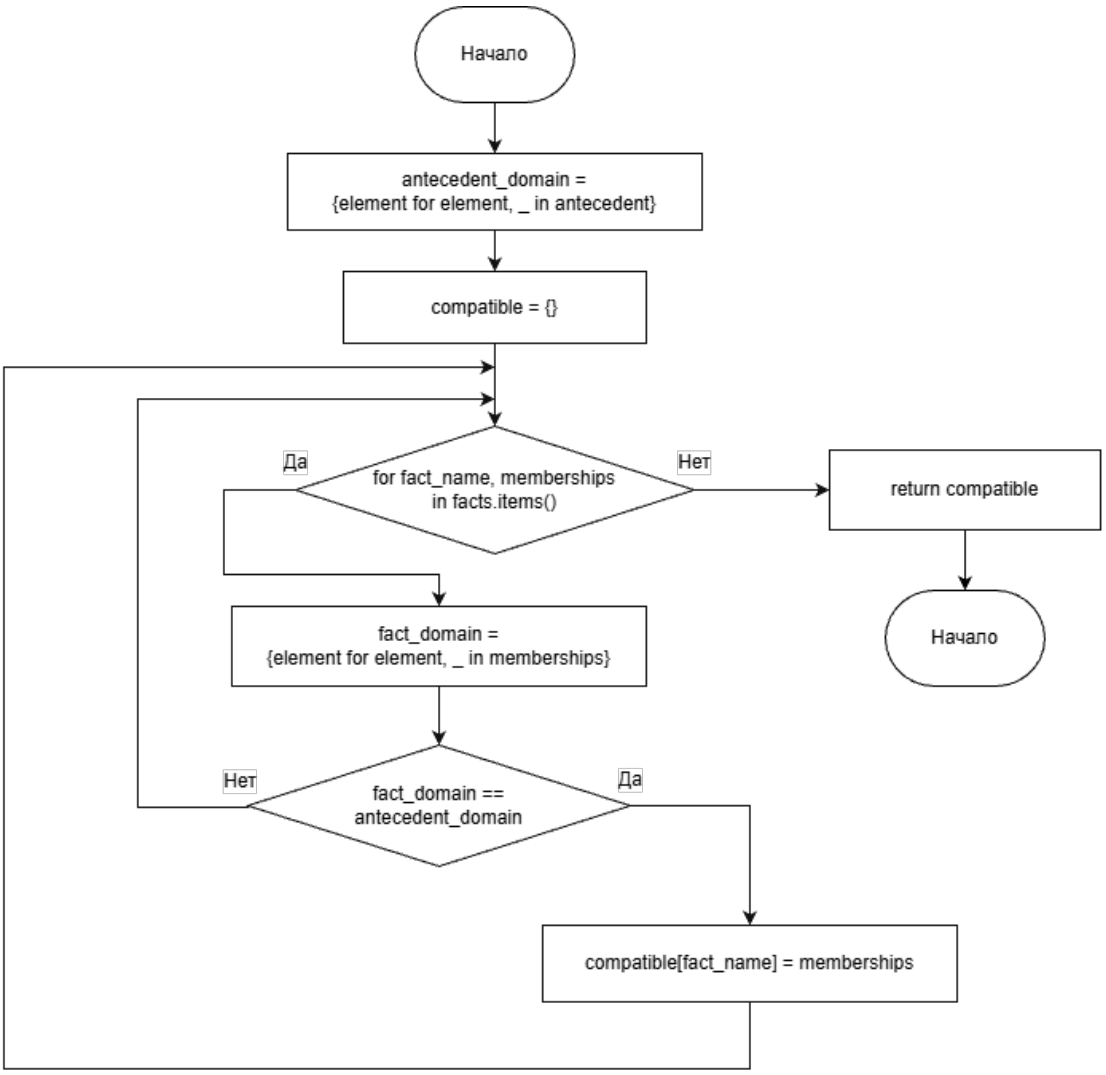


Рис. 5 Блок-схема функции `find_compatible_facts()`

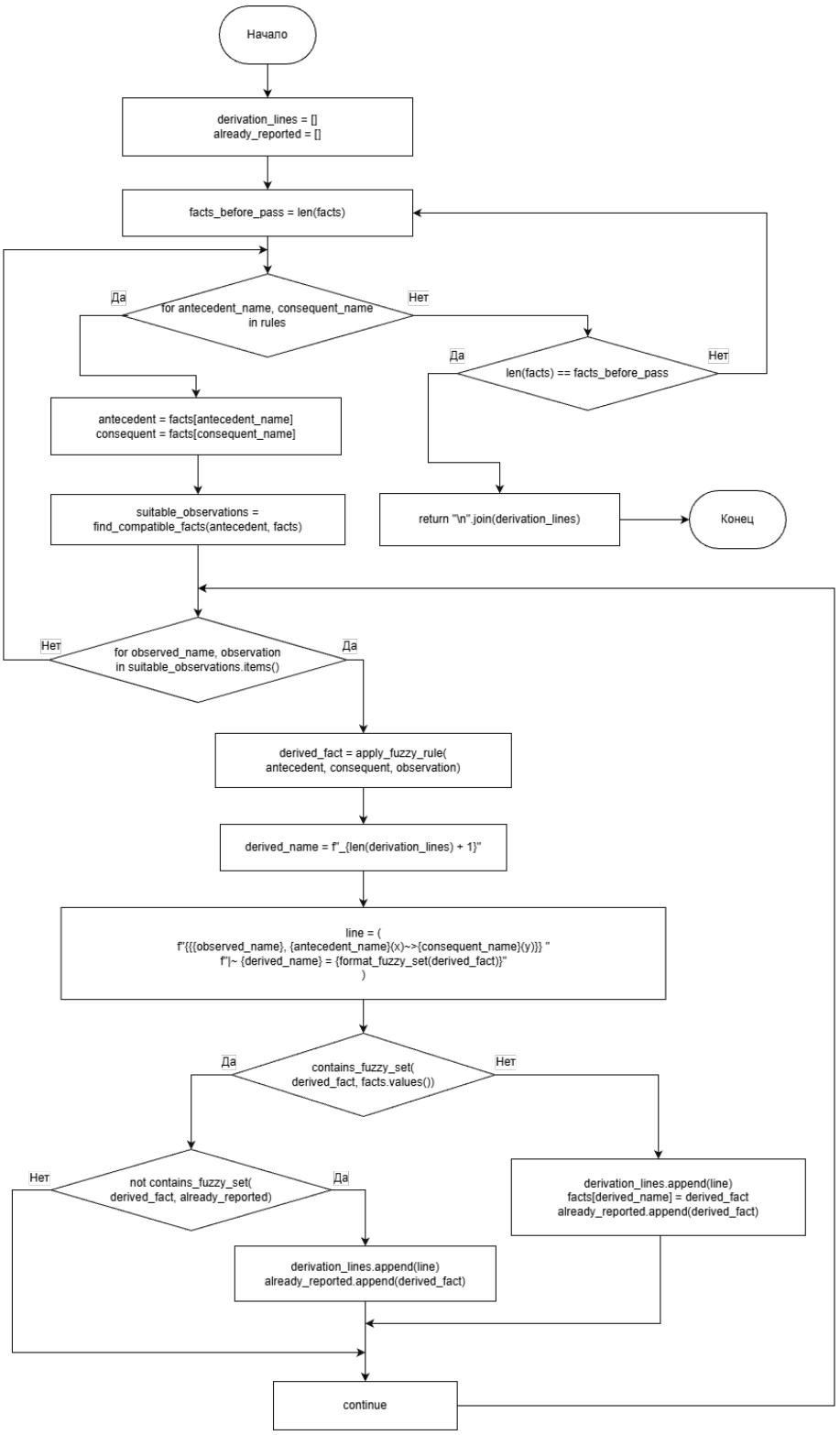


Рис. 6 Блок-схема функции `forward_fuzzy_inference()`

Демонстрация результатов работы программы

Программа `lab1_refactored.py` запущена с базой знаний из файла `input_original.txt`, приведённой выше.

Полный результат запуска:

```
{A, A(x)~>B(y)} |~ _1 = {<y1, 0.4>,<y2, 0.0>,<y3, 0.3>,<y4, 1.0>}
{C, A(x)~>B(y)} |~ _2 = {<y1, 1.0>,<y2, 0.7>,<y3, 1.0>,<y4, 1.0>}
{A, C(x)~>A(y)} |~ _3 = {<x1, 1.0>,<x2, 1.0>,<x3, 1.0>,<x4, 1.0>}
{C, C(x)~>A(y)} |~ _4 = {<x1, 0.0>,<x2, 0.3>,<x3, 1.0>,<x4, 0.5>}
{B, B(x)~>C(y)} |~ _5 = {<x1, 0.7>,<x2, 1.0>,<x3, 0.0>,<x4, 0.2>}
{_2, B(x)~>C(y)} |~ _6 = {<x1, 1.0>,<x2, 1.0>,<x3, 0.7>,<x4, 0.7>}
{_3, A(x)~>B(y)} |~ _7 = {<y1, 1.0>,<y2, 1.0>,<y3, 1.0>,<y4, 1.0>}
{_6, C(x)~>A(y)} |~ _8 = {<x1, 0.7>,<x2, 0.7>,<x3, 1.0>,<x4, 0.7>}
{_8, A(x)~>B(y)} |~ _9 = {<y1, 0.7>,<y2, 0.7>,<y3, 0.7>,<y4, 1.0>}
{_9, B(x)~>C(y)} |~ _10 = {<x1, 0.7>,<x2, 1.0>,<x3, 0.7>,<x4, 0.7>}
```

В строке `{факт, правило} |~ _k` указаны использованный факт, применённое правило и обозначение результата. Нумерация соответствует порядку вывода строк. Программа показывает результат, совпадающий с исходным фактом, один раз, но повторно в базу его не добавляет. Поэтому результаты `_1`, `_4` и `_5` совпадают соответственно с B, A и C и не сохраняются как отдельные факты.

На первом проходе получаются строки `_1`–`_6`. При применении правила $A(x) \to B(y)$ к факту A получено множество B. Например,

$$\mu_{B'}(y_1) = \max\{\min(0,1), \min(0.3,1), \\ \min(1,0.4), \min(0.5,0.4)\} = 0.4.$$

С наблюдением C правило $A(x) \to B(y)$ даёт `_2` со степенями $(1.0, 0.7, 1.0, 1.0)$.

На втором проходе добавляются `_7` и `_8`, на третьем — `_9` и `_10`. Четвёртый проход не расширяет базу, и вывод завершается. Напечатано 10 строк; добавлено 7 различных фактов.

Ответы на контрольные вопросы

1. При каких $A'(x)$ можно получить субнормальное множество $B'(y)$, если посылка и заключение правила нормальные?

При нормальном B существует y с $\mu_B(y) = 1$, а потому $I_G(A(x), B(y)) = 1$ для всех x. Из max-min композиции следует $h(B') = h(A')$. Следовательно, B' субнормально тогда и только тогда, когда A' субнормально.

2. При каких $A'(x)$ можно получить нормальное $B'(y)$, если посылка и заключение правила субнормальные?

Необходимо и достаточно существования таких x и y, что $\mu_{A'}(x) = 1$ и $\mu_A(x) \le \mu_B(y)$. Тогда $I_G(A(x), B(y)) = 1$ и $\mu_{B'}(y) = 1$.

3. Можно ли получить $B'(y) = B(y)$, если посылка и заключение нормальные?

Да. При $A' = A$ и нормальном A всегда получается $B' = B$. Однако совпадение $A' = A$ является достаточным, но не необходимым условием: другие наблюдения также могут дать то же заключение.

4. Можно ли получить $B'(y) = B(y)$, если посылка и заключение субнормальные?

Да. Например, при $A' = A$ выполняется равенство $\mu_{B'}(y) = \min\big(\mu_B(y), h(A)\big)$. Поэтому если $h(A) \ge h(B)$, результат совпадает с B.

5. Можно ли получить $B'(y) = B(y)$, если посылка нормальная, а заключение субнормальное?

Да. При $A' = A$ имеем $h(A) = 1$, поэтому $\mu_{B'}(y) = \min(\mu_B(y), 1) = \mu_B(y)$ для всех y.

6. Можно ли получить $B'(y) = B(y)$, если посылка субнормальная, а заключение нормальное?

При $A' = A$ — нельзя, так как тогда $h(B') \le h(A) < 1$, а $h(B) = 1$. Однако при произвольном $A' \ne A$ равенство возможно: верхняя граница результата определяется также входным наблюдением A'. Например, для $A = (0.2, 0.4)$, $B = (1, 0.3)$ и $A' = (0, 1)$ композиция по импликации Гёделя даёт $B' = B$.

Личный вклад

Сайковский А. В.: реализация max-min композиции и механизма прямого нечёткого вывода.

ВЫВОД

В ходе выполнения лабораторной работы разработана программа прямого нечёткого логического вывода на основе импликации Гёделя и t-нормы минимума. Реализованы построение нечёткого бинарного отношения, max-min композиция и последовательное применение правил к совместимым фактам. Реализован разбор имён предикатов из одной или двух буквенно-цифровых позиций с буквой в начале. Для базы знаний из файла `input_original.txt` получены 10 строк вывода; база расширена на 7 различных фактов. Результаты соответствуют вычислительной схеме обобщённого modus ponens. Приведены блок-схемы основных алгоритмов программы.

Использованные источники

1. Логические основы интеллектуальных систем. Практикум: учебно-методическое пособие / В. В. Голенков, В. П. Ивашенко, Д. Г. Колб, К. А. Уваров. — Минск: БГУИР, 2011.
2. Документация Python: модуль `re` (регулярные выражения), `https://docs.python.org/3/library/re.html`.
3. Исходный код лабораторной работы: `lab1_refactored.py`; входная база знаний: `input_original.txt`.